

APACHE SPARK

Lê Anh Quân



MỤC LỤC

I. Tổng quan về kiến trúc của Apache Spark

1. Apache Spark là gì?
2. Kiến trúc của Apache Spark
3. Spark Context/Spark Session
4. Tiến trình chạy một job của Apache Spark

II. Mô hình lập trình & khái niệm core

1. RDD
2. Transformation vs Action
3. Lazy Execution
4. DAG
5. Spark SQL/DataFrame/Dataset
6. Broadcast Variables & Accumulators

III. Cơ chế thực thi & tối ưu

1. Data Partitioning
2. Shuffling
3. Data Skew
4. Memory Spill

IV. Spark Streaming

1. Spark Streaming là gì?
2. Dstream vs Structured Streaming
3. Khả năng giải quyết bài toán streaming

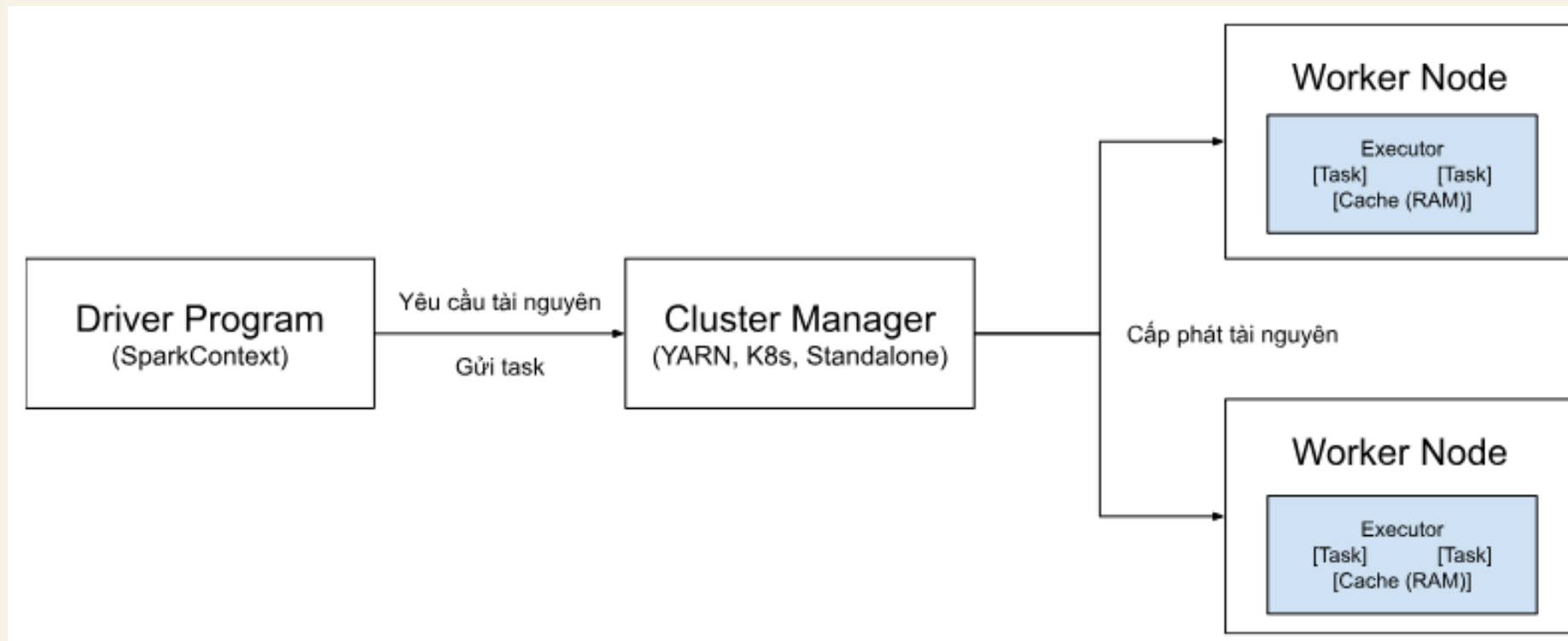
APACHE SPARK LÀ GÌ?

Apache Spark là một framework mã nguồn mở tính toán cụm. Tốc độ xử lý của Spark có được do việc tính toán được thực hiện ở bộ nhớ trong (in-memories) hay thực hiện hoàn toàn trên RAM. Spark cho phép xử lý dữ liệu theo thời gian thực, vừa nhận dữ liệu từ các nguồn khác nhau đồng thời thực hiện ngay việc xử lý trên dữ liệu vừa nhận được (Spark Streaming).

Spark không có hệ thống file của riêng mình, nó sử dụng hệ thống file khác nhau như: HDFS, Cassandra, S3,... Spark hỗ trợ nhiều kiểu định dạng file khác nhau (text, csv, json...) đồng thời nó hoàn toàn không phụ thuộc vào bất cứ một hệ thống file nào.



KIẾN TRÚC CỦA APACHE SPARK



SPARK CONTEXT VS SESSION

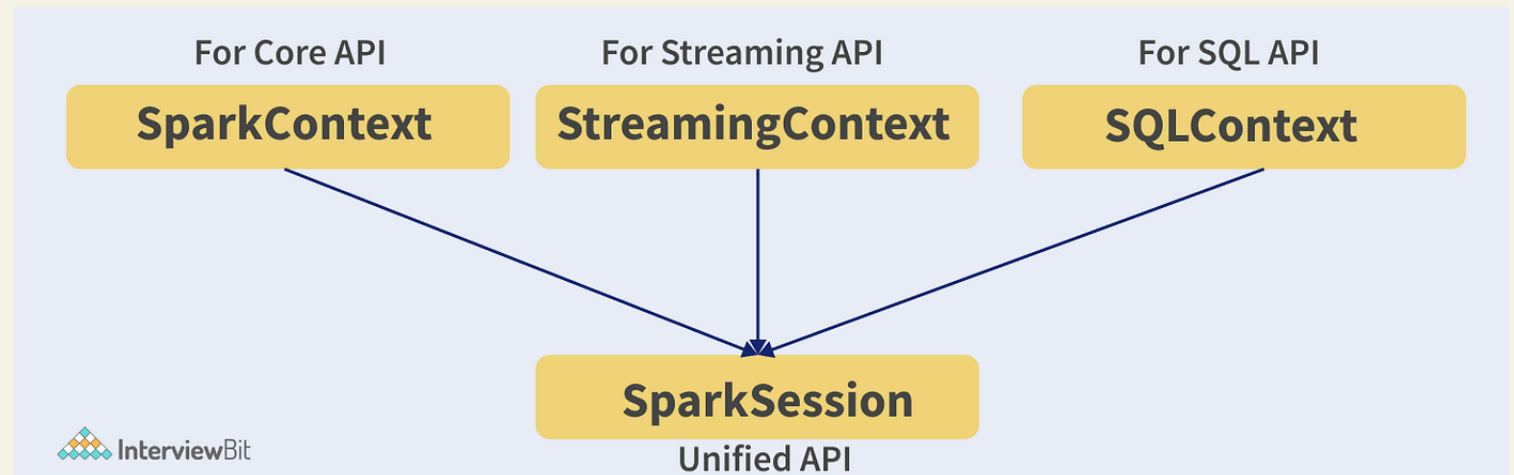
SPARK CONTEXT

SparkContext là cổng kết nối chính của ứng dụng Apache Spark với cụm máy chủ, giúp điều phối tài nguyên và tạo các tập dữ liệu phân tán (RDD). Chức năng chính của SparkContext bao gồm:

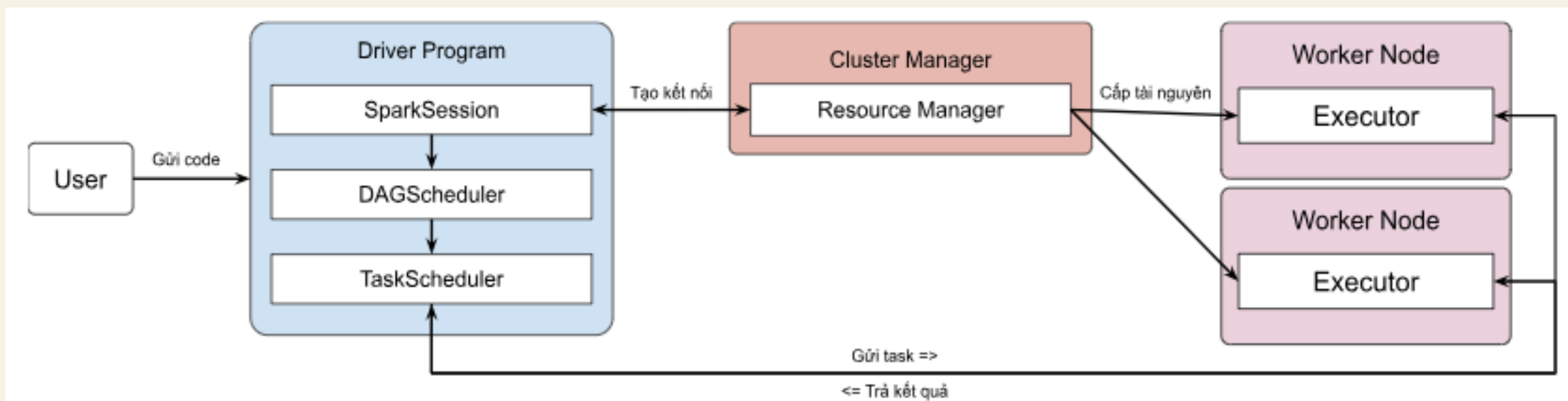
- Cổng kết nối máy chủ duy nhất
- Quản lý tài nguyên
- Tạo và quản lý RDD
- Quản lý biến chia sẻ

SPARK SESSION

Xuất hiện cùng với phiên bản Spark 2.0, SparkSession là điểm truy cập thống nhất (Unified Entry Point) cho mọi tính năng của Apache Spark. Nó đóng vai trò như một giao diện lập trình cấp cao giúp người dùng tương tác trực tiếp với dữ liệu có cấu trúc thông qua DataFrame, Dataset, các câu lệnh Spark SQL, hoặc các hệ thống lưu trữ như Hive.



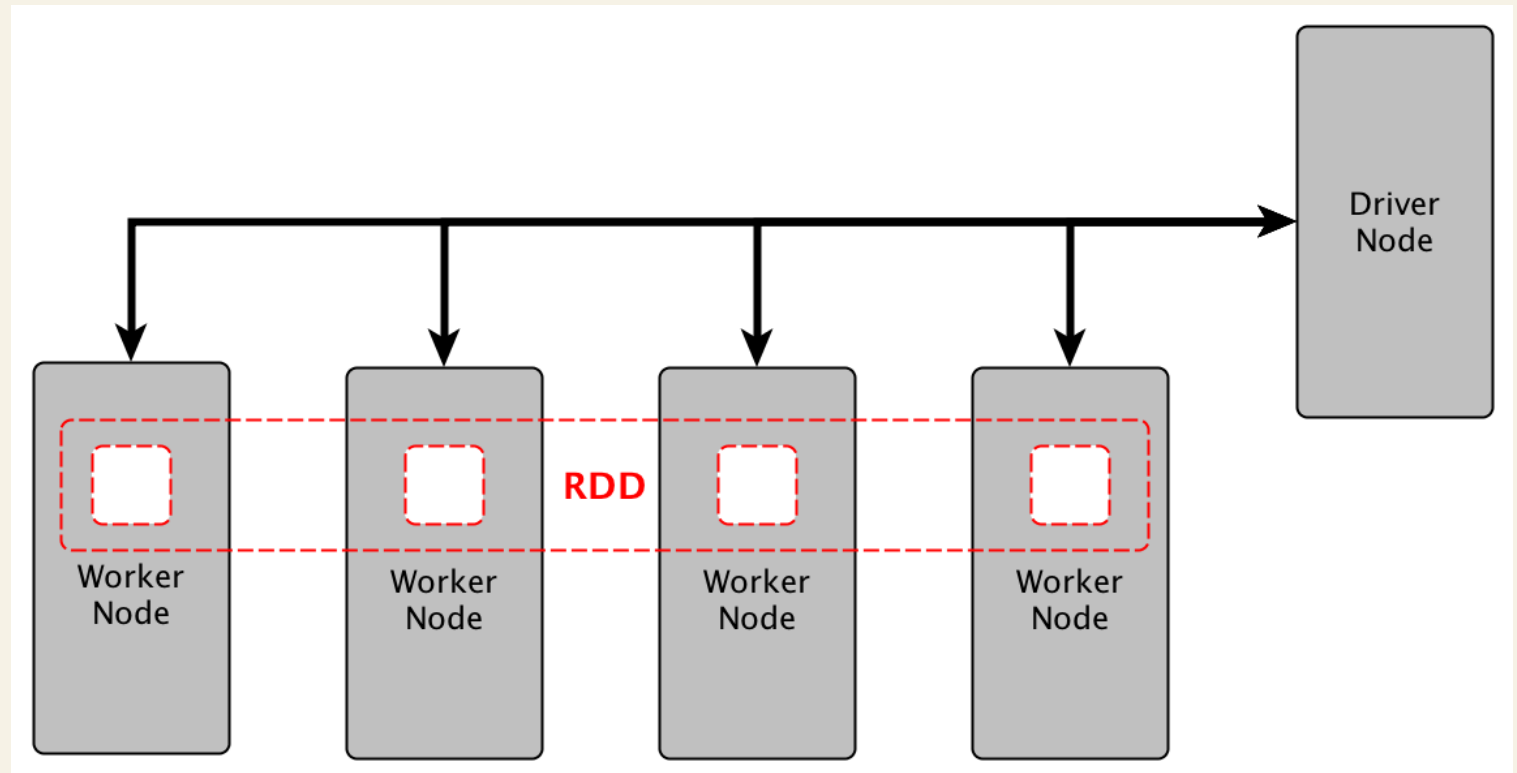
TIẾN TRÌNH JOB CỦA SPARK



RDD

RDD (Resilient Distributed Dataset) là cấu trúc dữ liệu cốt lõi trong Apache Spark - một tập hợp phần tử được phân tán trên nhiều máy trong cluster, cho phép xử lý song song. Về bản chất, RDD không chứa dữ liệu thật ngay lúc khai báo, mà là một công thức mô tả cách tạo ra dữ liệu đó (đọc từ đâu, biến đổi qua những bước nào). Các tính chất cốt lõi của RDD bao gồm:

- Bất biến (Immutable)
- Phân mảnh (Partitioned)
- Fault-tolerance nhờ Lineage
- Trì hoãn thực thi (Lazy)

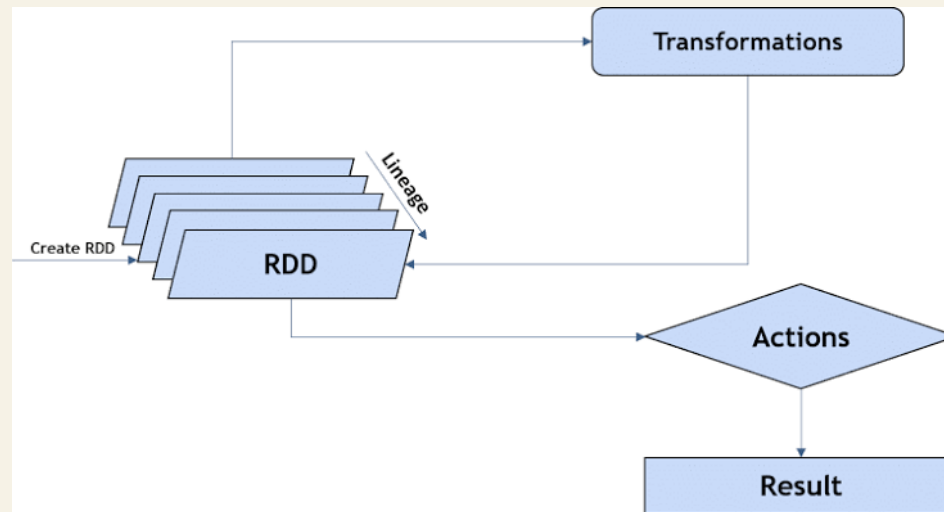


TRANSFORMATION VS ACTIONS

TRANSFORMATION

Transformation (phép biến đổi) trong Spark là các câu lệnh được dùng để xây dựng một RDD mới từ một hoặc nhiều RDD ban đầu. Đặc trưng cốt lõi của Transformation là cơ chế thực thi lười biếng (Lazy Evaluation).

Transformation được chia ra thành Narrow Transition hoặc Wide Transition dựa trên cách dữ liệu di chuyển giữa các phân vùng.



ACTIONS

Action (hành động) là thao tác kích hoạt quá trình thực thi thực tế của toàn bộ chuỗi các phép biến đổi (transformation) trước đó và trả về kết quả cho chương trình điều khiển (driver) hoặc ghi dữ liệu ra hệ thống lưu trữ. Khác với Transformation, Action buộc Spark phải xây dựng biểu đồ DAG, chia thành các stage và chạy các task trên cụm máy.

LAZY EVALUATION

Lazy Execution (hay lazy evaluation) trong Apache Spark là cơ chế trì hoãn việc thực thi các lệnh tính toán cho đến khi xuất hiện hành động (Action). Thay vì chạy ngay, Spark ghi nhận các bước biến đổi dữ liệu (Transformation) thành một sơ đồ logic gọi là DAG (Directed Acyclic Graph) để tối ưu hóa toàn bộ quá trình trước khi chạy. Các lợi ích Lazy Execution mang lại bao gồm:

- Tối ưu hóa hiệu năng
- Tiết kiệm tài nguyên
- Khả năng chịu lỗi cao

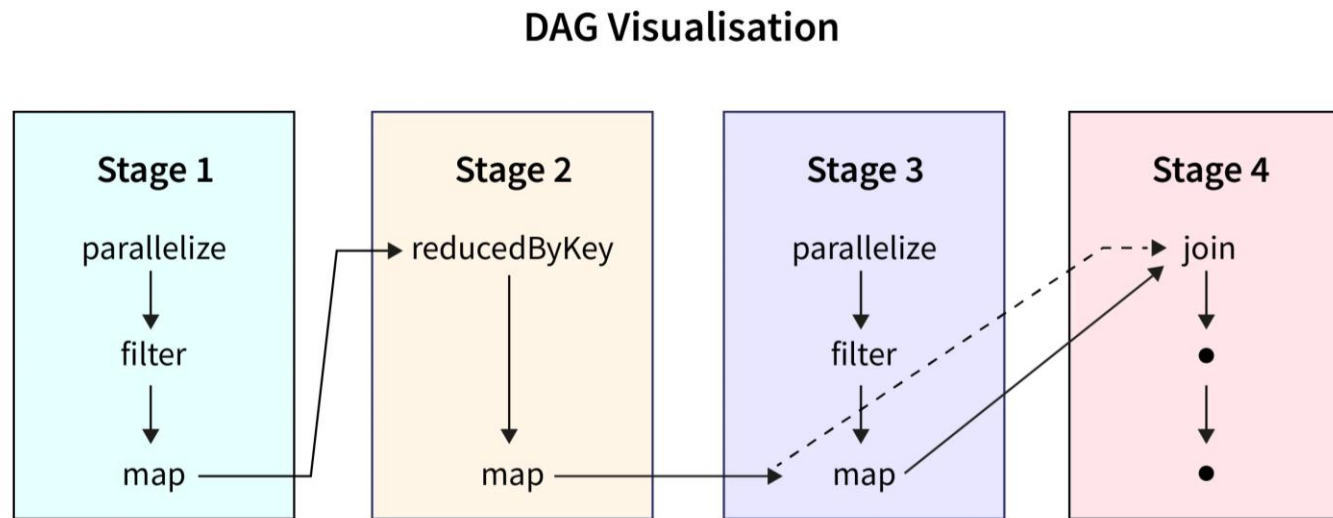
Lazy Evaluation

Deferred Execution Until Action



DAG

DAG dịch ra tiếng việt là đồ thị có hướng không chu trình. Đồ thị này trong Spark biểu diễn chuỗi logic các phép toán chuyển đổi dữ liệu (Transformations) trên RDD/DataFrame, đi từ nguồn dữ liệu ban đầu cho đến khi gặp một lệnh hành động (Action). Khi một Action kích hoạt DAG, DAGScheduler sẽ phân rã đồ thị này thành các thành phần nhỏ hơn theo thứ tự phân cấp: DAG => Stages => Tasks.

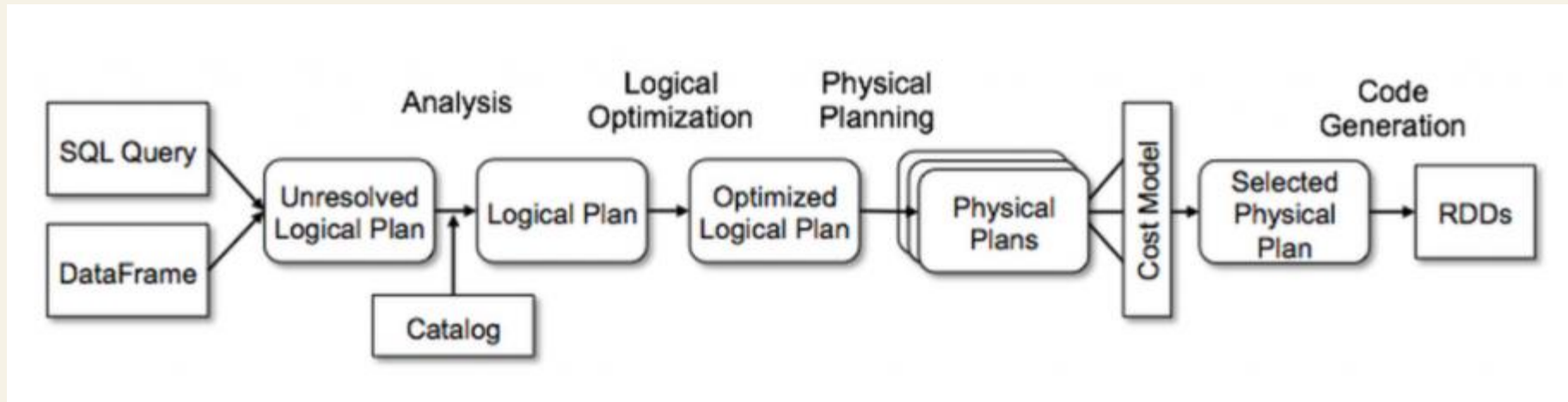


SPARK SQL/DATAFRAME/DATASET

Trong khi Spark SQL, DataFrame và Dataset là 3 khái niệm khác nhau, với Spark SQL là ngôn ngữ khai báo, DataFrame và Dataset là bảng dữ liệu phân tán không kiểm tra/kiểm tra dữ liệu, cả 3 được kết nối và hợp nhất với nhau bởi Catalyst Optimizer và Project Tungsten.

Catalyst Optimizer là bộ não tối ưu cốt lõi của Spark SQL, DataFrame và Dataset. Bản chất của nó là một trình tối ưu hóa truy vấn (Query Optimizer) có nhiệm vụ tự động chuyển đổi các câu lệnh được viết thành một kế hoạch thực thi bằng mã máy (Java Bytecode) sao cho tốn ít tài nguyên và chạy nhanh nhất có thể.

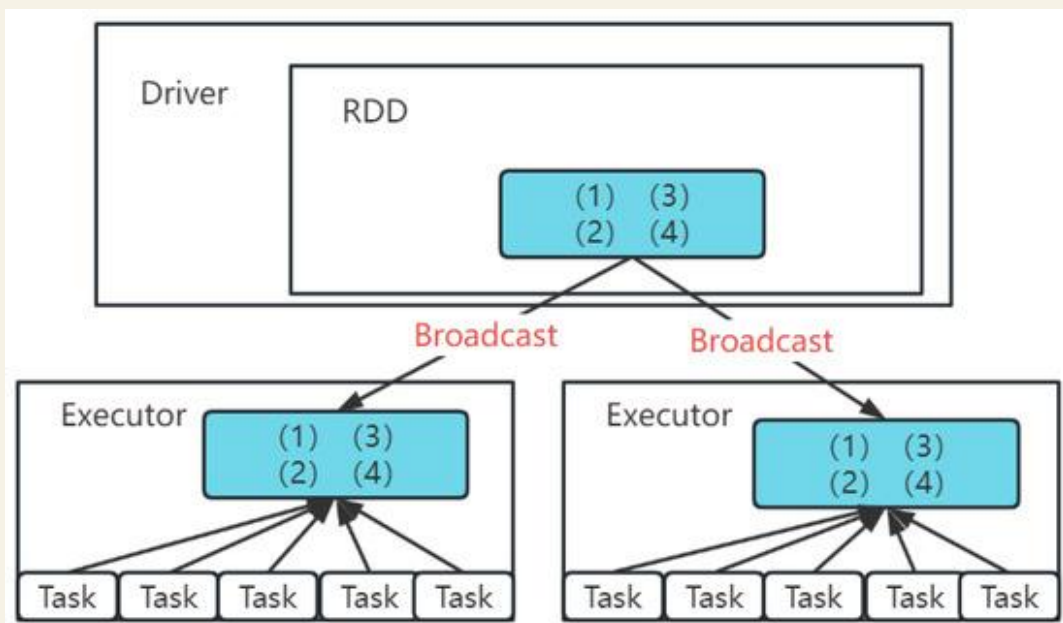
Nếu Catalyst Optimizer đóng vai trò là bộ não thì **Project Tungsten** chính là cơ bắp của Spark. Tập trung hoàn toàn vào việc tối ưu hóa hiệu suất phần cứng xuống mức thấp nhất, Project Tungsten sinh ra để loại bỏ các nút thắt cổ chai về bộ nhớ và CPU.



BROADCAST VARIABLES & ACCUMULATORS

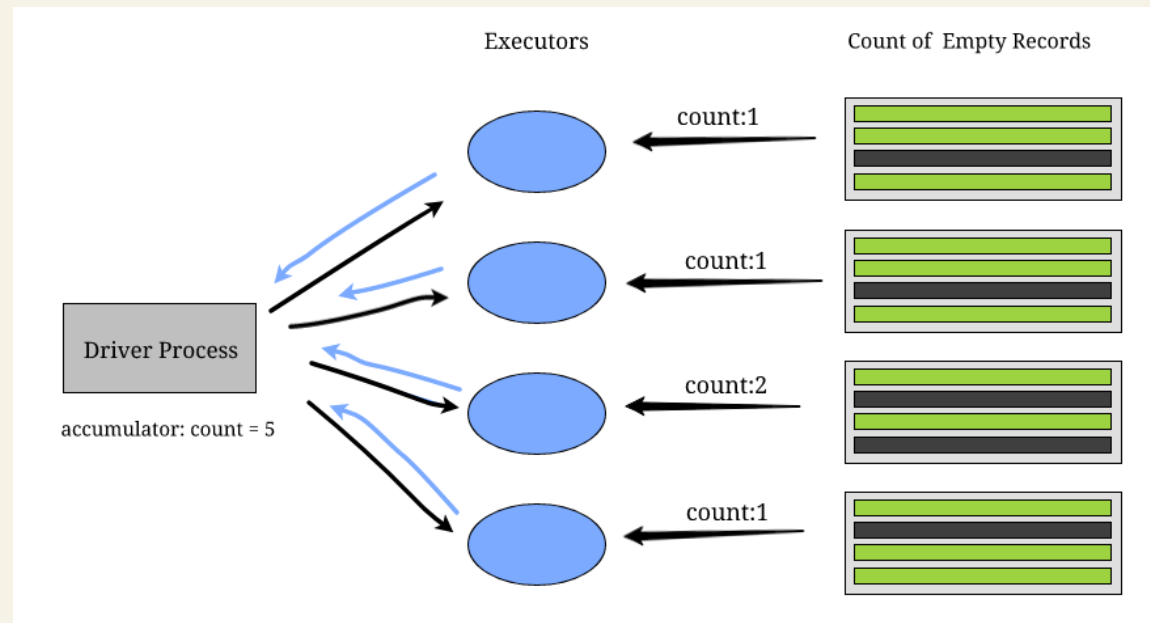
BROADCAST VARIABLES

Broadcast Variables là biến cho phép lập trình viên lưu trữ một bản sao dữ liệu chỉ đọc (Read-Only) duy nhất trên mỗi máy con (Worker Node), thay vì gửi kèm dữ liệu đó cùng với mọi Task.



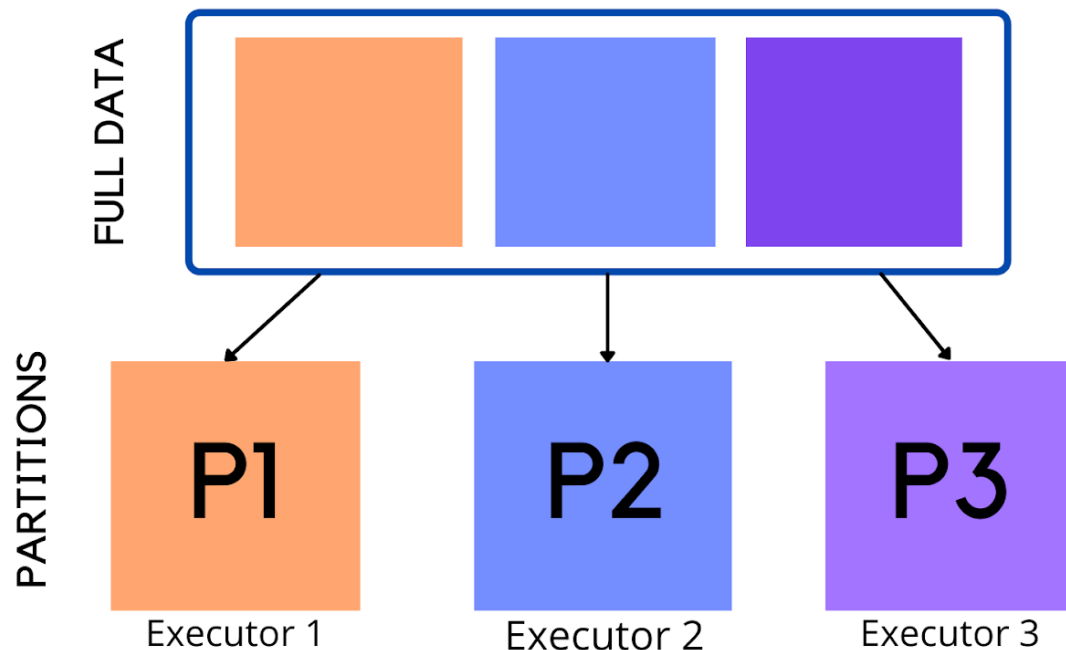
ACCUMULATORS

Accumulators là các biến chỉ có thể "cộng dồn" (Write-Only từ phía máy con) thông qua một phép toán có tính chất giao hoán và kết hợp (như cộng, đếm), tương tự như biến toàn cục (Global Variable) nhưng hoạt động trên môi trường phân tán.



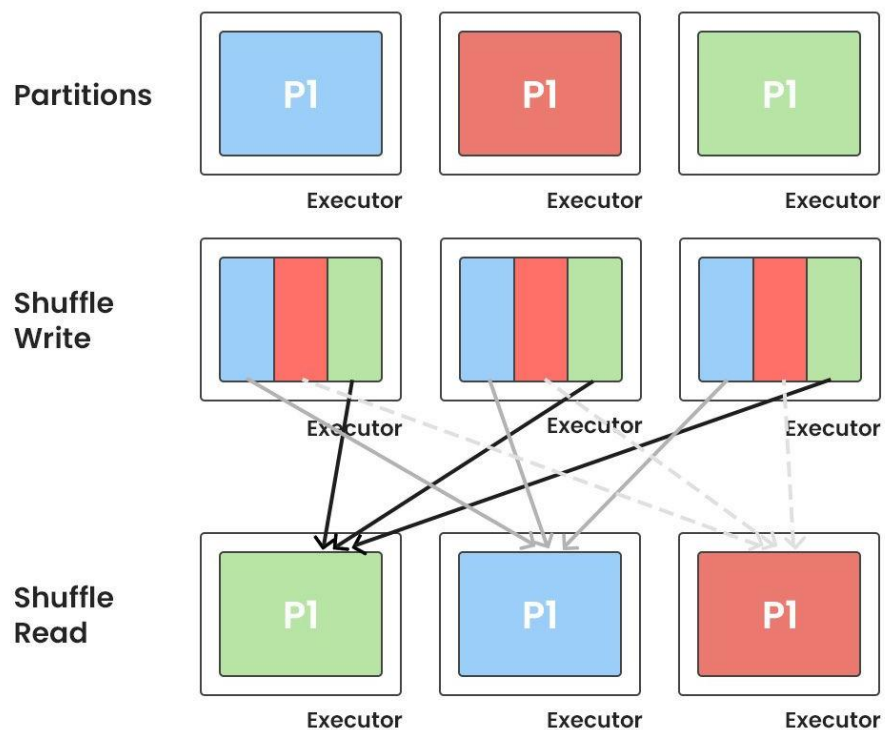
DATA PARTITIONING

Data Partitioning trong Spark là hành động chia nhỏ một tập dữ liệu lớn (RDD, DataFrame) thành các phần nhỏ hơn (Partitions), được lưu trữ trên bộ nhớ RAM của các nút khác nhau trong cụm máy tính để có thể xử lý song song. Điểm mấu chốt khiến hệ thống này của Spark đặc biệt là tính chất In-Memory và Ephemeral.



SHUFFLING

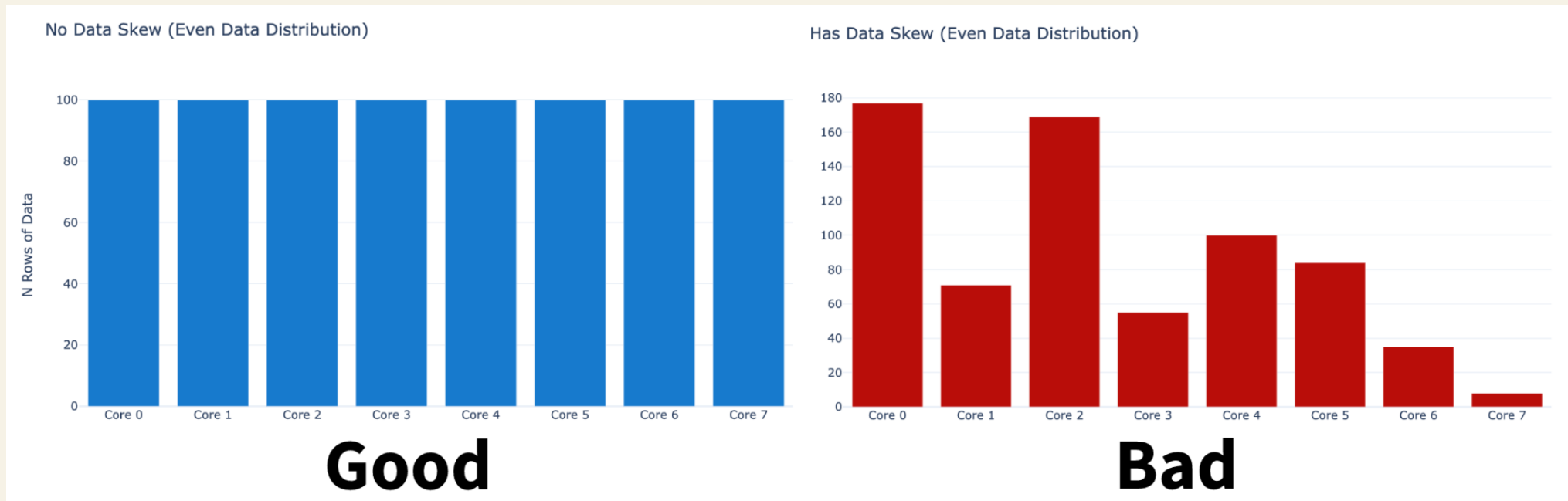
How Spark Architecture Shuffle Works ?



Shuffling trong Apache Spark là quá trình tái phân phối dữ liệu giữa các phân vùng (Partitions) trên toàn bộ cụm máy tính. Đây là hành động di chuyển dữ liệu qua mạng internet giữa các nút con (Executors) để gom các bản ghi có cùng một đặc điểm hoặc khóa về cùng một chỗ. Shuffling cũng chính là ranh giới để Spark tách các Stage trong đồ thị DAG, chỉ xảy ra khi thực hiện các phép toán Wide Transformations.

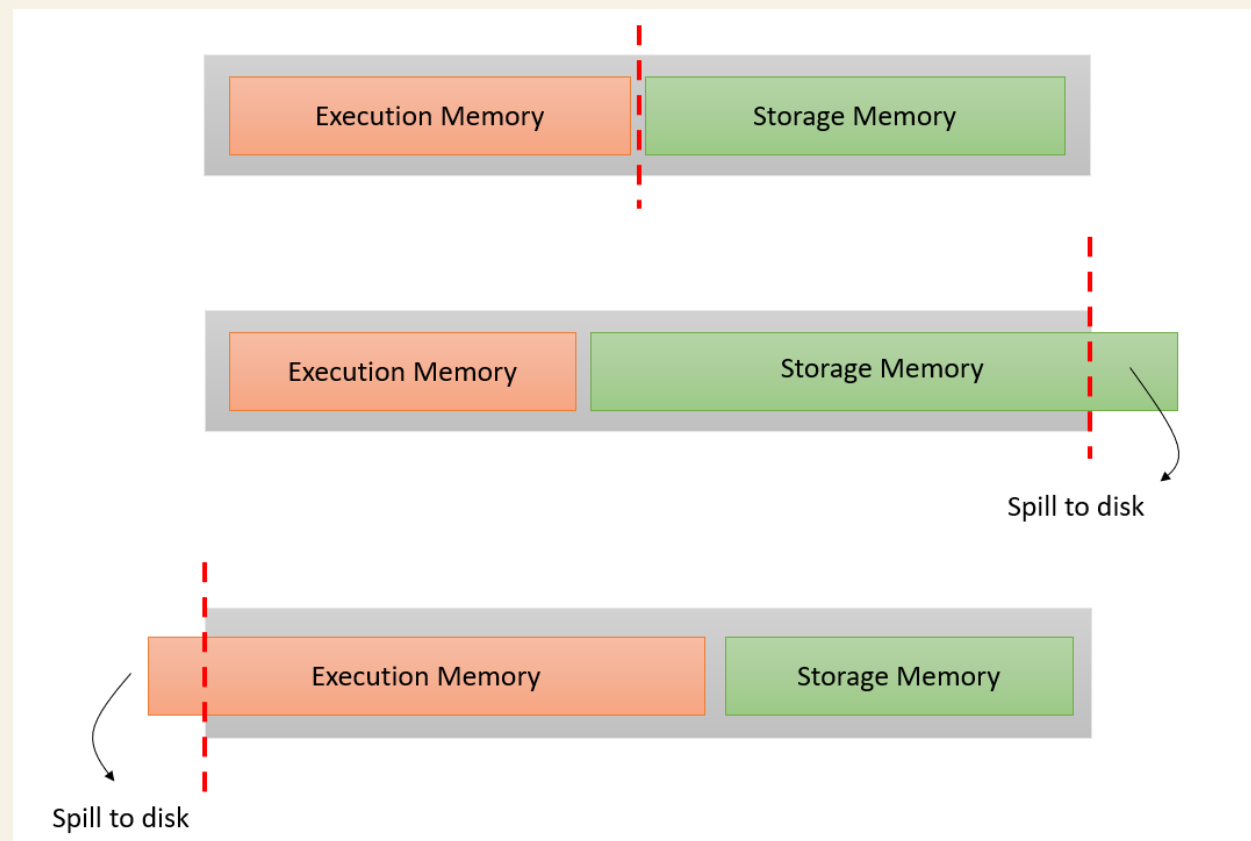
DATA SKEW

Data Skew thường phát sinh sau quá trình Shuffling (Gom nhóm dữ liệu qua mạng). Khi Spark dùng hàm Hash để gom các dữ liệu có cùng một Từ khóa (Key) về chung một chỗ, nếu một Key xuất hiện quá nhiều lần, phân vùng chứa Key đó sẽ bị phình to. Các giải pháp xử lý Data Skew trong thực tế bao gồm sử dụng Broadcast Join (nếu có thể), kỹ thuật Salting, loại bỏ các giá trị Null hay kích hoạt tính năng AQE.



MEMORY SPILL

Memory Spill (Tràn bộ nhớ) là hiện tượng xảy ra khi một Partition hoặc các tập dữ liệu trung gian trong một Task quá lớn, vượt quá dung lượng RAM được cấp phát cho Task đó. Khi RAM bị cạn kiệt, để tránh việc chương trình bị sụp đổ hoàn toàn do lỗi Out Of Memory (OOM), Spark buộc phải chọn giải pháp "cứu cánh": Xả bớt dữ liệu tạm thời từ RAM xuống ổ đĩa cứng cục bộ (Local Disk), sau đó đọc ngược lại lên RAM khi cần tính toán tiếp.



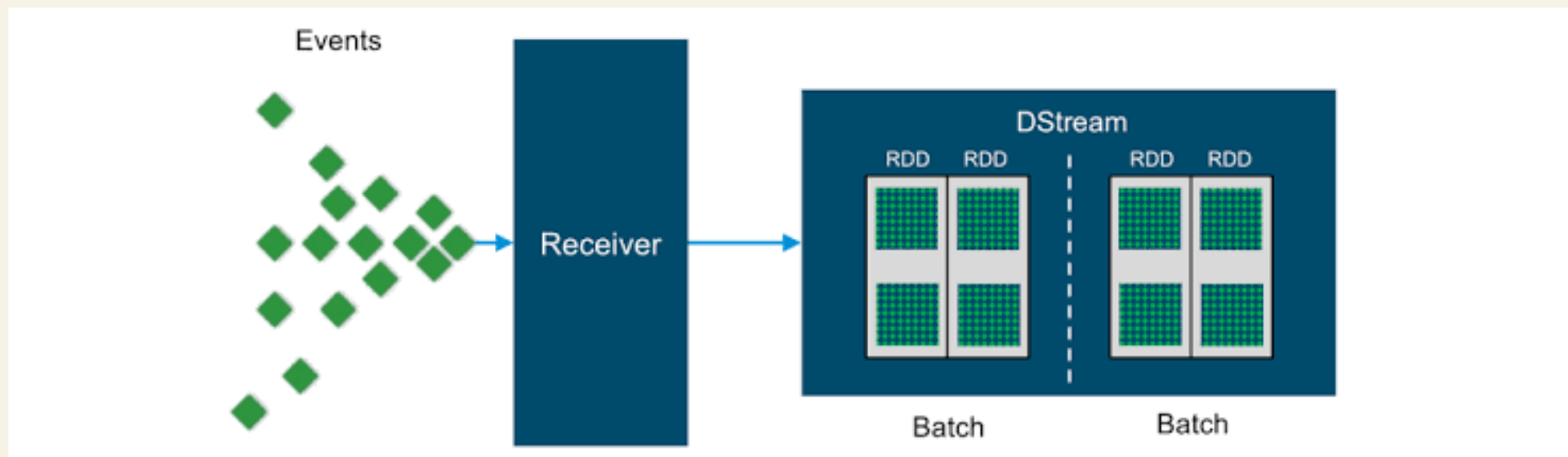
SPARK STREAMING LÀ GÌ?

Spark Streaming là một thành phần mở rộng của lõi Apache Spark, cho phép xử lý các luồng dữ liệu trực tiếp (live data streams) theo cơ chế micro-batch: thay vì xử lý từng bản ghi ngay khi nó đến, Spark gom dữ liệu trong một khoảng thời gian ngắn gọi là batch interval (ví dụ mỗi 1 giây), sau đó xử lý cả khoảng đó như một batch job nhỏ. Nhờ vậy Spark Streaming vẫn tận dụng được engine xử lý batch mạnh mẽ sẵn có của Spark, đạt độ trễ thấp gần với thời gian thực (near real-time) chứ không phải xử lý tức thời từng sự kiện một.



DSTREAM (DISCRETIZED STREAM)

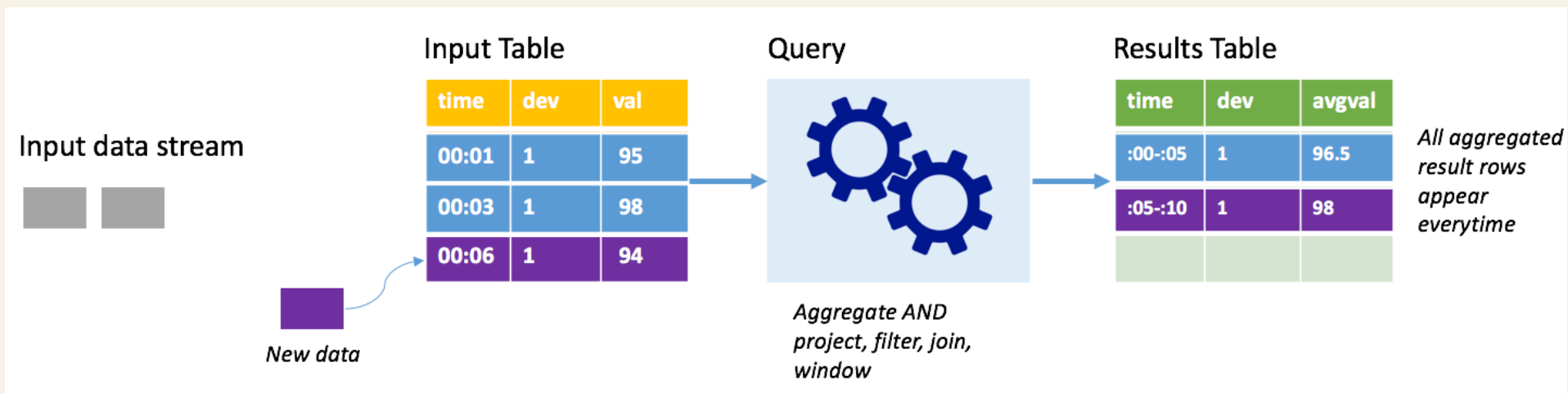
Hoạt động theo cơ chế micro-batching đã nêu ở trên, DStream API chia nhỏ luồng dữ liệu liên tục theo batch interval, mỗi khoảng được nạp vào Spark dưới dạng một tập hợp các RDD thô. Vì dùng RDD nên nó không tận dụng được Catalyst Optimizer và Project Tungsten. Ngoài ra, DStream chỉ nhận biết dữ liệu theo Processing Time — tức thời điểm Spark thực sự nhận được bản ghi — mà không biết đến thời điểm sự kiện thực sự xảy ra ở nguồn. Điều này khiến việc gom nhóm dữ liệu theo cửa sổ thời gian (windowing) dễ bị sai lệch khi dữ liệu đến trễ so với thứ tự thực tế.



STRUCTURED STREAMING

Ra đời từ Spark 2.0, mô hình này coi luồng dữ liệu trực tiếp như một bảng dữ liệu mở rộng vô hạn (Unbounded Table). Dữ liệu mới đến sẽ liên tục được ghi thêm vào dòng cuối của bảng (append). Những lợi ích khi sử dụng Structured Streaming bao gồm:

- Sử dụng chung API với DataFrame và Spark SQL
- Tận dụng được Catalyst Optimizer và Project Tungsten
- Hỗ trợ xử lý dữ liệu đến muộn (Late Data) bằng Watermarking
- Continuous Processing mode



KHẢ NĂNG GIẢI QUYẾT BÀI TOÁN STREAMING

Apache Spark giải quyết bài toán dữ liệu bằng cơ chế micro-batch kết hợp với Continuous Processing, mang lại sự cân bằng tối ưu giữa hiệu năng cao, khả năng chịu lỗi và tính nhất quán dữ liệu. So với các giải pháp xử lý luồng chuyên dụng khác như Apache Flink, Apache Kafka Streams, hay Storm, Spark có thể mạnh tuyệt đối về hệ sinh thái tích hợp nhưng lại có độ trễ cao hơn một chút trong cấu hình mặc định.

